

MySQL

What is SQL?

SQL is the standard language for dealing with Relational Databases.

SQL is used to insert, search, update, and delete database records.

The MySQL SELECT Statement:

The SELECT statement is used to select data from a database.

The data returned is stored in a result table, called the result-set.

The following SQL selects the "CustomerName", "City", and "Country" columns from the "Customers" table:

```
SELECT CustomerName, City, Country FROM Customers;
```

SELECT Syntax

```
SELECT column1, column2, ...  
FROM table_name;
```

To select all columns, without specifying every column name, use the following syntax:

```
SELECT * FROM table_name;
```

MySQL SELECT DISTINCT Statement

The SELECT DISTINCT statement is used to return only distinct (unique) values.

In a table, a column may contain several duplicate values - and sometimes you want to list only the unique values.

SELECT DISTINCT Country FROM Customers;

SELECT DISTINCT Syntax

```
SELECT DISTINCT column1, column2, ...  
FROM table_name;
```

MySQL WHERE Clause:

The WHERE clause is used to filter records.

The WHERE clause is used to extract only those records that fulfill a specific condition.

Example:

```
SELECT * FROM Customers  
WHERE Country = 'Mexico';
```

WHERE Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Note: The WHERE clause is not only used in SELECT statements, it is also used in UPDATE, DELETE, etc.

MySQL ORDER BY Keyword:

The ORDER BY keyword is used to sort the result-set in ascending or descending order.

The ORDER BY keyword sorts the records in ascending order (ASC) by default.

Example:

Sort the products from lowest to highest price:

```
SELECT * FROM Products  
ORDER BY Price;
```

ORDER BY Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
ORDER BY column1, column2, ... ASC|DESC;
```

MySQL AND Operator:

The WHERE clause can contain one or many AND operators.

The AND operator is used to filter records based on more than one condition.

Note: The AND operator displays a record if **all** the conditions are TRUE.

Example:

```
SELECT * FROM Customers  
WHERE Country = 'UK' AND City = 'London';
```

AND Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition1 AND condition2 AND condition3 ...;
```

MySQL OR Operator:

The WHERE clause can contain one or more OR operators.

The OR operator is used to filter records based on more than one condition.

Note: The OR operator displays a record if any of the conditions are TRUE.

The following SQL selects all customers where city is "Berlin" OR "Stuttgart":

Example:

```
SELECT * FROM Customers
WHERE City = 'Berlin' OR City = 'Stuttgart';
```

OR Syntax:

```
SELECT column1, column2, ...
FROM table_name
WHERE condition1 OR condition2 OR condition3 ...;
```

MySQL NOT Operator

The NOT operator is used in the WHERE clause to return all records that DO NOT match the specified criteria. It reverses the result of a condition from true to false and vice-versa.

The following SQL selects all customers that are NOT from Germany:

Example:

```
SELECT * FROM Customers
WHERE NOT Country = 'Germany';
```

The **NOT** operator in SQL is used to **exclude records** that match a condition. It returns the opposite of a condition.

Suppose we have an **Employees** table:

EmpID	Name	Department	Salary	City	Bonus
101	Alice	HR	30000	Chennai	2000

EmpID	Name	Department	Salary	City	Bonus
102	Bob	IT	45000	Salem	NULL
103	Charlie	Sales	35000	Coimbatore	1500
104	David	IT	50000	Madurai	NULL
105	Eva	HR	28000	Chennai	1000

1. NOT LIKE

Used to **exclude values matching a pattern**.

Syntax

```
SELECT * FROM Employees
WHERE Name NOT LIKE 'A%';
```

Explanation

- A% means names starting with **A**.
- NOT LIKE returns names **not starting with A**.

Output

Name

Bob

Charlie

David

Eva

2. NOT BETWEEN

Used to **exclude values within a range**.

Syntax

```
SELECT * FROM Employees
WHERE Salary NOT BETWEEN 30000 AND 45000;
```

Explanation

- Between 30000 and 45000 includes:
 - 30000
 - 35000
 - 45000
- NOT BETWEEN returns salaries outside this range.

Output

Name Salary

David 50000

Eva 28000

3. NOT IN

Used to **exclude specific values**.

Syntax

```
SELECT * FROM Employees
WHERE Department NOT IN ('HR', 'Sales');
```

Explanation

Exclude employees from HR and Sales.

Output

Name Department

Bob IT

David IT

4. IS NOT NULL

Used to find rows where a column **contains a value** (not NULL).

Syntax

```
SELECT * FROM Employees  
WHERE Bonus IS NOT NULL;
```

Explanation

Returns employees who have a bonus.

Output

Name	Bonus
------	-------

Alice	2000
-------	------

Charlie	1500
---------	------

Eva	1000
-----	------

5. NOT EXISTS

Used with a **subquery**. It returns rows only when the subquery returns **no records**.

Suppose another table:

Projects

ProjectID	EmpID
-----------	-------

P101	101
------	-----

P102	103
------	-----

Query

```
SELECT Name  
FROM Employees E  
WHERE NOT EXISTS (  
    SELECT *  
    FROM Projects P  
    WHERE P.EmpID = E.EmpID  
);
```

Explanation

Employees 101 and 103 are assigned to projects.

NOT EXISTS returns employees **not assigned to any project**.

Output

Name

Bob

David

Eva

Quick difference

- NOT LIKE → Excludes a text pattern.
- NOT BETWEEN → Excludes a range of values.
- NOT IN → Excludes listed values.
- IS NOT NULL → Returns rows where the value exists.
- NOT EXISTS → Returns rows with no matching record in another table.

The MySQL INSERT INTO Statement

The INSERT INTO statement is used to insert new records in a table.

It is possible to write the INSERT INTO statement in two ways:

Syntax 1

Specify both the column names and the values to be inserted:

```
INSERT INTO table_name (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

Syntax 2

If you insert values for ALL the columns of the table, you can omit the column names.

However, the order of the values must be in the same order as the columns in the table:


```
INSERT INTO table_name
VALUES (value1, value2, value3, ...);
```

MySQL NULL Values

What is a NULL Value?

If a field in a table is optional, it is possible to insert or update a record without adding any value to this field. This way, the field will be saved with a NULL value.

A NULL value represents an unknown, missing, or inapplicable data in a database field. It is not a value itself, but a placeholder to indicate the absence of data.

Note: A NULL value is different from zero (0) or an empty string (""). A field with a NULL value is one that has been left blank upon record creation.

How to Test for NULL Values?

It is not possible to test for NULL values with comparison operators, such as =, <, or <>.

We will have to use the IS NULL and IS NOT NULL operators instead.

IS NULL Syntax

```
SELECT column_names
FROM table_name
WHERE column_name IS NULL;
```

IS NOT NULL Syntax

```
SELECT column_names
FROM table_name
WHERE column_name IS NOT NULL;
```

The MySQL UPDATE Statement

The UPDATE statement is used to update or modify one or more records in a table.

UPDATE Syntax

UPDATE *table_name*

SET *column1 = value1, column2 = value2, ...*

WHERE *condition*;

Note: Be careful when updating records in a table! Notice the WHERE clause in the UPDATE statement. The WHERE clause specifies which record(s) that should be updated. If you omit the WHERE clause, all records in the table will be updated!

UPDATE Table

The following SQL updates the record with CustomerID = 1, with a new contact person AND a new city.

Example:

UPDATE Customers

SET ContactName = 'Alfred Schmidt', City = 'Frankfurt'

WHERE CustomerID = 1;

The selection from the "Customers" table will now look like this

UPDATE Multiple Records

The WHERE clause determines which records that will be updated.

Example:

UPDATE Customers

SET PostalCode = 00000

WHERE Country = 'Mexico';

The MySQL DELETE Statement

The DELETE statement is used to delete existing records in a table.

DELETE Syntax

DELETE FROM *table_name* WHERE *condition*;

Note: Be careful when deleting records in a table! Notice the WHERE clause in the DELETE statement. The WHERE clause specifies which record(s) should be deleted. If you omit the WHERE clause, all records in the table will be deleted!

Delete All Records

It is possible to delete all records in a table, without deleting the table. This means that the table structure, attributes, and indexes will be intact.

DELETE FROM *table_name*;

Delete a Table

To delete the table completely, use the DROP TABLE statement:

DROP TABLE *table_name*;

The MySQL LIMIT Clause

The LIMIT clause is used to limit the number of records to return.

The LIMIT clause is useful on large tables with thousands of records. Returning a large number of records can impact performance.

LIMIT Syntax

SELECT *column_name(s)*
FROM *table_name*
WHERE *condition*
LIMIT *number*;

Example:

The following SQL selects the first three records from the "Customers" table:

Example

```
SELECT * FROM Customers  
LIMIT 3;
```

The following SQL says "return only 3 records, start on record 4 (OFFSET 3)":

Example

```
SELECT * FROM Customers  
LIMIT 3 OFFSET 3;
```

MySQL Aggregate Functions

An aggregate function is a function that performs a calculation on a set of values, and returns a single value.

Aggregate functions are often used with the GROUP BY clause of the SELECT statement.

The GROUP BY clause splits the result-set into groups of values and the aggregate function can be used to return a single value for each group.

The most commonly used SQL aggregate functions are:

- **MIN()** - returns the smallest value of a column
- **MAX()** - returns the largest value of a column
- **COUNT()** - returns the number of rows in a set
- **SUM()** - returns the sum of a numerical column
- **AVG()** - returns the average value of a numerical column

MySQL MIN() Function

The MIN() function returns the smallest value of the selected column.

The MIN() function works with numeric, string, and date data types.

MIN() Syntax

```
SELECT MIN(column_name)  
FROM table_name  
WHERE condition;
```

Example:

```
SELECT MIN(Price) AS SmallestPrice  
FROM Products;
```

MySQL MAX() Function

The MAX() function returns the largest value of the selected column.

The MAX() function works with numeric, string, and date data types.

MAX() Syntax

```
SELECT MAX(column_name)  
FROM table_name  
WHERE condition;
```

MySQL COUNT() Function

The COUNT() function returns the number of rows that matches a specified criterion.

COUNT() Syntax

```
SELECT COUNT([DISTINCT] column_name | *)  
FROM table_name  
WHERE condition;
```

The behavior of COUNT() depends on the argument used within the parentheses:

- **COUNT(*)** - Counts the total number of rows in a table (including NULL values).
- **COUNT(columnname)** - Counts all non-null values in the column.
- **COUNT(DISTINCT columnname)** - Counts only the unique, non-null values in the column.

MySQL SUM() Function:

The SUM() function is used to calculate the total sum of values within a numeric column.

The SUM() function ignores NULL values in the column.

SUM() Syntax

```
SELECT SUM(column_name)  
FROM table_name  
WHERE condition;
```

MySQL AVG() Function:

The AVG() function returns the average value of a numeric column.

The AVG() function ignores NULL values in the column.

AVG() Syntax

```
SELECT AVG(column_name)  
FROM table_name  
WHERE condition;
```

The MySQL LIKE Operator

The LIKE operator is used in a WHERE clause to search for a specified pattern within a column's text data.

There are two wildcards often used in conjunction with the LIKE operator:

- A percent sign % - represents zero, one, or multiple characters
- A underscore sign _ - represents a single character

The percent sign and the underscore can also be used in combinations!

LIKE Syntax

```
SELECT column1, column2, ...  
FROM table_name  
WHERE columnN LIKE pattern;
```

The MySQL IN Operator

The IN operator is used in the WHERE clause to check if a specified column's value matches any value within a provided list.

The IN operator functions as a shorthand for multiple OR conditions, making queries shorter and more readable.

IN Syntax

```
SELECT column_name(s)  
FROM table_name  
WHERE column_name IN (value1, value2, ...);
```

The NOT IN Operator

By using the NOT IN operator, you return all records that are NOT any of the values in the list.

The following SQL selects all customers that are NOT located in "Germany", "France" or "UK":

Example

```
SELECT * FROM Customers  
WHERE Country NOT IN ('Germany', 'France', 'UK');
```

The MySQL BETWEEN Operator

The BETWEEN operator is used in the WHERE clause to select values within a specified range.

The range is inclusive - the beginning and end values of the range are included in the results.

The values can be numbers, text, or dates.

BETWEEN Syntax

```
SELECT column_name(s)  
FROM table_name  
WHERE column_name BETWEEN value1 AND value2;
```

BETWEEN Example:

The following SQL selects all products with a price between 10 and 20:

Example:

```
SELECT * FROM Products  
WHERE Price BETWEEN 10 AND 20;
```

NOT BETWEEN

The NOT BETWEEN operator is used in the WHERE clause to select values outside a specified range.

The following SQL returns all products with a price NOT between 10 and 20:

Example

```
SELECT * FROM Products  
WHERE Price NOT BETWEEN 10 AND 20;
```

MySQL Aliases

An alias is created with the AS keyword, and is often used to make a column name more readable.

An alias only exists for the duration of that query.

Alias for Column Syntax:

```
SELECT column_name AS alias_name  
FROM table_name;
```

Alias with Spaces

If you want your alias to contain one or more spaces, like "Contact Person", surround the aliasname with single or double quotes:

Example

```
SELECT CustomerName AS Customer, ContactName AS "Contact Person"  
FROM Customers;
```


Alias for Tables

The following SQL selects all the orders from the customer with CustomerID=4 (Around the Horn). We use the "Customers" and "Orders" tables, and give them the table aliases of "c" and "o" respectively (Here we use aliases to make the SQL shorter):

Example

```
SELECT o.OrderID, o.OrderDate, c.CustomerName  
FROM Customers AS c, Orders AS o  
WHERE c.CustomerName='Around the Horn' AND c.CustomerID=o.CustomerID;
```

